Introduction to Haskell Programming

Data structures – Binary trees

S P Suresh

Chennai Mathematical Institute

Haskell @ CMI

Binary trees

- A binary tree data structure is defined as follows:

Binary trees

- A binary tree data structure is defined as follows:
- The empty tree is a binary tree

Binary trees

- A binary tree data structure is defined as follows:
- The empty tree is a binary tree
- A node containing an element with left and right subtrees is a binary tree

Binary trees

- A binary tree data structure is defined as follows:
- The empty tree is a binary tree
- A node containing an element with left and right subtrees is a binary tree
- Type constructor `BTree`

```
data BTree a = Nil | Node (BTree a) a (BTree a)
```

Binary trees

- A binary tree data structure is defined as follows:
- The empty tree is a binary tree
- A node containing an element with left and right subtrees is a binary tree
- Type constructor `BTree`

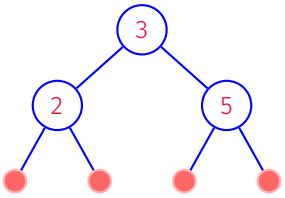
```
data BTree a = Nil | Node (BTree a) a (BTree a)
```

- Two value constructors:

```
Nil :: BTree a
Node :: BTree a → a → BTree a → BTree a
```

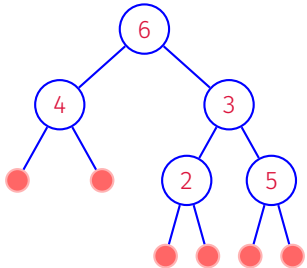
Binary trees

```
Node (Node Nil 2 Nil) 3
     (Node Nil 5 Nil)
```



Binary trees

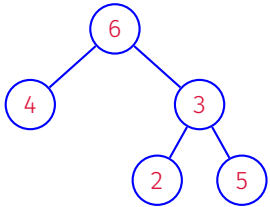
```
Node (Node Nil 4 Nil) 6
    (Node (Node Nil 2 Nil) 3
        (Node Nil 5 Nil))
```



Binary trees

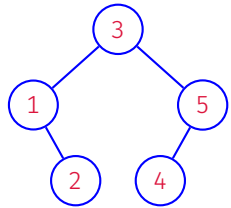
```
Node (Node Nil 4 Nil) 6
    (Node (Node Nil 2 Nil) 3
        (Node Nil 5 Nil))
```

- We omit nodes representing `Nil` usually



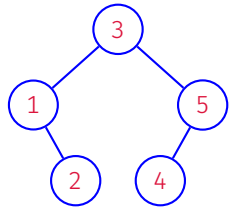
Binary trees

- Yet another binary tree



Binary trees

- Yet another binary tree



- Corresponding `BTree`

```
Node
    (Node Nil 1 (Node Nil 2 Nil))
3
    (Node (Node Nil 4 Nil) 5 Nil)
```

Functions on binary trees

- Basic functions on trees

`isNil :: BTree a → Bool`	Check if a tree is empty
`size :: BTree a → Integer`	Number of nodes in a tree
`height :: BTree a → Integer`	Number of nodes on longest path from root to a leaf

Functions on binary trees

- Basic functions on trees

`isNil :: BTree a → Bool`	Check if a tree is empty
`size :: BTree a → Integer`	Number of nodes in a tree
`height :: BTree a → Integer`	Number of nodes on longest path from root to a leaf

- The definitions

```
isNil Nil    = True          size Nil            = 0
isNil _      = False         size (Node tl x tr) = 1 + size tl + size tr

height Nil            = 0
height (Node tl x tr) = 1 + max (height tl) (height tr)
```

Functions on binary trees

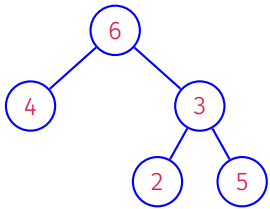
- Reflect the tree on its “vertical axis”

Functions on binary trees

- Reflect the tree on its “vertical axis”

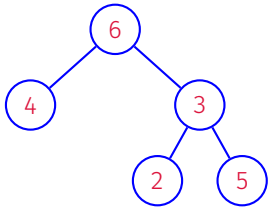
Functions on binary trees

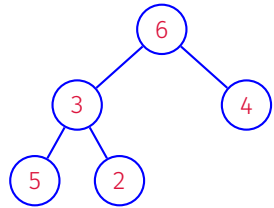
- Reflect the tree on its “vertical axis”



Functions on binary trees

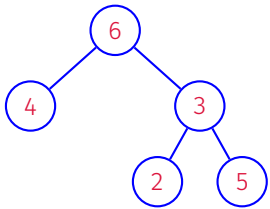
- Reflect the tree on its “vertical axis”

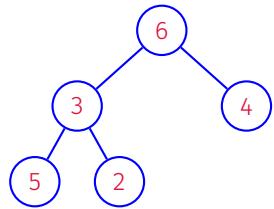




Functions on binary trees

- Reflect the tree on its “vertical axis”





- Haskell code:

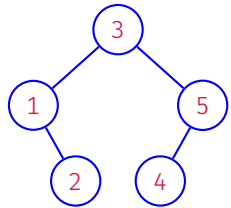
```
reflect :: BTree a → BTree a
reflect Nil            = Nil
reflect (Node tl x tr) = Node (reflect tr) x (reflect tl)
```

Functions on binary trees

- `levels` – List nodes level by level and from left to right inside each level

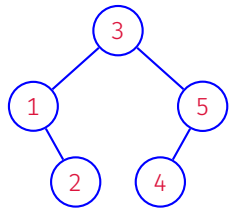
Functions on binary trees

- `levels` – List nodes level by level and from left to right inside each level
- Let `t` be the tree below:



Functions on binary trees

- `levels` – List nodes level by level and from left to right inside each level
- Let `t` be the tree below:



- `levels t = [3,1,5,2,4]`

Functions on binary trees

- `levels` using a queue data structure

```
levels t    = go (empty +: t) where
    (+:) :: Queue (BTree a) → BTree a → Queue (BTree a)
    q +: t  = if isNil t then q else enqueue t q

    go :: Queue (BTree a) → [a]
    go q    = if isEmpty q then [] else x: go (q' +: tl +: tr) where
        (Node tl x tr, q')  = dequeue q
```

Inorder listing

- `levels` lists all elements of a tree according to their levels

Inorder listing

- `levels` lists all elements of a tree according to their levels
- Another important way to render a tree as a list

Inorder listing

- `levels` lists all elements of a tree according to their levels
- Another important way to render a tree as a list
  - Recursively list all elements of the left subtree

Inorder listing

- `levels` lists all elements of a tree according to their levels
- Another important way to render a tree as a list
  - Recursively list all elements of the left subtree
  - List the root after that

Inorder listing

- `levels` lists all elements of a tree according to their levels
- Another important way to render a tree as a list
  - Recursively list all elements of the left subtree
  - List the root after that
  - Recursively list all elements of the right subtree

Inorder listing

- `levels` lists all elements of a tree according to their levels
- Another important way to render a tree as a list
  - Recursively list all elements of the left subtree
  - List the root after that
  - Recursively list all elements of the right subtree
- **Inorder traversal**

```
inorder :: BTree a → [a]
inorder Nil             = []
inorder (Node tl x tr)  = inorder tl ++ [x] ++ inorder tr
```

Efficiency of `inorder`

- Let $T(n)$ denote the number of steps needed to list a tree of size n

Efficiency of `inorder`

- Let $T(n)$ denote the number of steps needed to list a tree of size n
- Let n_l and n_r be the sizes of the left and right subtrees, respectively

Efficiency of `inorder`

- Let $T(n)$ denote the number of steps needed to list a tree of size n
- Let n_l and n_r be the sizes of the left and right subtrees, respectively
- $T(n) = T(n_l) + T(n_r) + n_l$

Efficiency of `inorder`

- Let $T(n)$ denote the number of steps needed to list a tree of size n
- Let n_l and n_r be the sizes of the left and right subtrees, respectively
- $T(n) = T(n_l) + T(n_r) + n_l$
  - Recall that `l1 ++ l2` takes time proportional to the length of `l1`

Efficiency of `inorder`

- Let $T(n)$ denote the number of steps needed to list a tree of size n
- Let n_l and n_r be the sizes of the left and right subtrees, respectively
- $T(n) = T(n_l) + T(n_r) + n_l$
  - Recall that `l1 ++ l2` takes time proportional to the length of `l1`
- Worst case is a **left-skewed tree**

Efficiency of `inorder`

- Let $T(n)$ denote the number of steps needed to list a tree of size n
- Let n_l and n_r be the sizes of the left and right subtrees, respectively
- $T(n) = T(n_l) + T(n_r) + n_l$
  - Recall that `l1 ++ l2` takes time proportional to the length of `l1`
- Worst case is a **left-skewed tree**
  - $n_l = n - 1$ and $n_r = 0$, and the same is true recursively

Efficiency of `inorder`

- Let $T(n)$ denote the number of steps needed to list a tree of size n
- Let n_l and n_r be the sizes of the left and right subtrees, respectively
- $T(n) = T(n_l) + T(n_r) + n_l$
  - Recall that `l1 ++ l2` takes time proportional to the length of `l1`
- Worst case is a **left-skewed tree**
  - $n_l = n - 1$ and $n_r = 0$, and the same is true recursively
  - $T(0) = 0$

Efficiency of `inorder`

- Let $T(n)$ denote the number of steps needed to list a tree of size n
- Let n_l and n_r be the sizes of the left and right subtrees, respectively
- $T(n) = T(n_l) + T(n_r) + n_l$
  - Recall that `l1 ++ l2` takes time proportional to the length of `l1`
- Worst case is a **left-skewed tree**
  - $n_l = n - 1$ and $n_r = 0$, and the same is true recursively
  - $T(0) = 0$
  - $T(n) = T(n-1) + (n-1)$

Efficiency of `inorder`

- Let $T(n)$ denote the number of steps needed to list a tree of size n
- Let n_l and n_r be the sizes of the left and right subtrees, respectively
- $T(n) = T(n_l) + T(n_r) + n_l$
  - Recall that `l1 ++ l2` takes time proportional to the length of `l1`
- Worst case is a **left-skewed tree**
  - $n_l = n - 1$ and $n_r = 0$, and the same is true recursively
  - $T(0) = 0$
  - $T(n) = T(n-1) + (n-1)$
  - **Solution**: $T(n) = \frac{n(n-1)}{2}$

Efficiency of `inorder`

- Let $T(n)$ denote the number of steps needed to list a tree of size n
- Let n_l and n_r be the sizes of the left and right subtrees, respectively
- $T(n) = T(n_l) + T(n_r) + n_l$
  - Recall that `l1 ++ l2` takes time proportional to the length of `l1`
- Worst case is a **left-skewed tree**
  - $n_l = n - 1$ and $n_r = 0$, and the same is true recursively
  - $T(0) = 0$
  - $T(n) = T(n-1) + (n-1)$
  - **Solution**: $T(n) = \frac{n(n-1)}{2}$
- The culprit is `(++)`

Efficiency of `inorder`

- Let $T(n)$ denote the number of steps needed to list a tree of size n
- Let n_l and n_r be the sizes of the left and right subtrees, respectively
- $T(n) = T(n_l) + T(n_r) + n_l$
  - Recall that `l1 ++ l2` takes time proportional to the length of `l1`
- Worst case is a **left-skewed tree**
  - $n_l = n - 1$ and $n_r = 0$, and the same is true recursively
  - $T(0) = 0$
  - $T(n) = T(n-1) + (n-1)$
  - **Solution**: $T(n) = \frac{n(n-1)}{2}$
- The culprit is `(++)`
- Can we avoid it?

More efficient `inorder`

- Define `go t l = inorder t ++ l`

More efficient `inorder`

- Define `go t l = inorder t ++ l`
- If we can define `go` independently, then `inorder t = go t []`

More efficient `inorder`

- Define `go t l = inorder t ++ l`
- If we can define `go` independently, then `inorder t = go t []`
- Let us calculate the definition of `go`

```
go Nil l
= inorder Nil ++ l = [] ++ l = l

go (Node tl x tr) l
= inorder (Node tl x tr) ++ l
= (inorder tl ++ [x] ++ inorder tr) ++ l
= inorder tl ++ ([x] ++ (inorder tr ++ l))
= inorder tl ++ x:go tr l
= go tl (x:go tr l)
```

More efficient `inorder`

- Final code

```
inorder :: BTree a → [a]
inorder t = go t [] where
    go Nil            l = l
    go (Node tl x tr) l = go tl (x:go tr l)
```

More efficient `inorder`

- Final code

```
inorder :: BTree a → [a]
inorder t = go t [] where
    go Nil            l = l
    go (Node tl x tr) l = go tl (x:go tr l)
```

- Let $T(n)$ be the number of steps required to compute `go t l`, where the size of `t` is n (the length of `l` does not matter!)

More efficient `inorder`

- Final code

```
inorder :: BTree a → [a]
inorder t = go t [] where
    go Nil            l = l
    go (Node tl x tr) l = go tl (x:go tr l)
```

- Let $T(n)$ be the number of steps required to compute `go t l`, where the size of `t` is n (the length of `l` does not matter!)
- Let n_l and n_r be the sizes of `tl` and `tr`

More efficient `inorder`

- Final code

```
inorder :: BTree a → [a]
inorder t = go t [] where
    go Nil            l = l
    go (Node tl x tr) l = go tl (x:go tr l)
```

- Let $T(n)$ be the number of steps required to compute `go t l`, where the size of `t` is n (the length of `l` does not matter!)
- Let n_l and n_r be the sizes of `tl` and `tr`
- $n = n_l + n_r + 1$

More efficient `inorder`

- Final code

```
inorder :: BTree a → [a]
inorder t = go t [] where
    go Nil            l = l
    go (Node tl x tr) l = go tl (x:go tr l)
```

- Let $T(n)$ be the number of steps required to compute `go t l`, where the size of `t` is n (the length of `l` does not matter!)
- Let n_l and n_r be the sizes of `tl` and `tr`
- $n = n_l + n_r + 1$
- $T(n) = T(n_l) + T(n_r) + 1$ (We add 1 as the cost of `(:)`)

More efficient `inorder`

- Final code

```
inorder :: BTree a → [a]
inorder t = go t [] where
    go Nil            l = l
    go (Node tl x tr) l = go tl (x:go tr l)
```

- Let $T(n)$ be the number of steps required to compute `go t l`, where the size of `t` is n (the length of `l` does not matter!)
- Let n_l and n_r be the sizes of `tl` and `tr`
- $n = n_l + n_r + 1$
- $T(n) = T(n_l) + T(n_r) + 1$ (We add 1 as the cost of `(:)`)
- **Solution**: $T(n) = n$

Creating a binary tree

- Create a binary tree from a list

Creating a binary tree

- Create a binary tree from a list
- As **balanced** as possible

Creating a binary tree

- Create a binary tree from a list
- As **balanced** as possible
- **Strategy**:

Creating a binary tree

- Create a binary tree from a list
- As **balanced** as possible
- **Strategy**:
  - Split the list in two halves

Creating a binary tree

- Create a binary tree from a list
- As **balanced** as possible
- **Strategy**:
  - Split the list in two halves
  - Recursively create a binary tree from each half

Creating a binary tree

- Create a binary tree from a list
- As **balanced** as possible
- **Strategy**:
  - Split the list in two halves
  - Recursively create a binary tree from each half
  - Join them together

Creating a binary tree

- Creating a balanced tree from a list

```
createTree :: [a] → BTree a
createTree [] = Nil
createTree xs = Node tl y tr where
    m               = length xs `div` 2
    (tl, y:ys)      = (createTree (take m xs), drop m xs)
    tr              = createTree ys

levels (createTree [0..14]) = [7,3,11,1,5,9,13,0,2,4,6,8,10,12,14]
height (createTree [0..14]) = 4
```

Creating a binary tree

- `length`, `take` and `drop` take linear time

Creating a binary tree

- `length`, `take` and `drop` take linear time
- $T(n) = 2T(n/2) + O(n)$ and hence $T(n) = O(n \log n)$

Creating a binary tree

- `length`, `take` and `drop` take linear time
- $T(n) = 2T(n/2) + O(n)$ and hence $T(n) = O(n \log n)$
- Can we improve?

Creating a binary tree

- `length`, `take` and `drop` take linear time
- $T(n) = 2T(n/2) + O(n)$ and hence $T(n) = O(n \log n)$
- Can we improve?
  - Culprit is the repeated use of `length`, `take` and `drop`

Creating a binary tree

- `length`, `take` and `drop` take linear time
- $T(n) = 2T(n/2) + O(n)$ and hence $T(n) = O(n \log n)$
- Can we improve?
  - Culprit is the repeated use of `length`, `take` and `drop`
  - Problem remains even if we use `splitAt`

Creating a binary tree

- `length`, `take` and `drop` take linear time
- $T(n) = 2T(n/2) + O(n)$ and hence $T(n) = O(n \log n)$
- Can we improve?
  - Culprit is the repeated use of `length`, `take` and `drop`
  - Problem remains even if we use `splitAt`
  - Can we avoid that?

Deriving a more efficient `createTree`

- Avoid `length` in recursive calls

Deriving a more efficient `createTree`

- Avoid **`length`** in recursive calls
- New version of `createTree`

```
createTree :: [a] → BTree a
createTree xs        = ct (length xs) xs where
    ct 0 xs          = Nil
    ct n xs          = Node tl y tr where
        m            = n `div` 2
        (tl, y:ys)   = (ct m (take m xs), drop m xs)
        tr           = ct (n-m-1) ys
```

Deriving a more efficient `createTree`

- Avoid **`length`** in recursive calls
- New version of `createTree`

```
createTree :: [a] → BTree a
createTree xs       = ct (length xs) xs where
    ct 0 xs         = Nil
    ct n xs         = Node tl y tr where
        m           = n `div` 2
        (tl, y:ys)  = (ct m (take m xs), drop m xs)
        tr          = ct (n-m-1) ys
```

- Let `go n xs = (ct n (take n xs), drop n xs)`

Deriving a more efficient `createTree`

- Avoid **`length`** in recursive calls
- New version of `createTree`

```
createTree :: [a] → BTree a
createTree xs        = ct (length xs) xs where
    ct 0 xs          = Nil
    ct n xs          = Node tl y tr where
        m            = n `div` 2
        (tl, y:ys)   = (ct m (take m xs), drop m xs)
        tr           = ct (n-m-1) ys
```

- Let `go n xs = (ct n (take n xs), drop n xs)`
- Then `createTree xs = fst (go (length xs) xs)`

Deriving a more efficient `createTree`

- Calculating a direct definition of `go`, assuming `0 < n ⩽ length xs`

```
go n xs
= (ct n (take n xs), drop n xs)
= (Node tl y tr,     drop n xs) where
    m            = n `div` 2
    (tl, y:rest) = (ct m (take m (take n xs)), drop m (take n xs))
    tr           = ct (n-m-1) rest
```

Deriving a more efficient `createTree`

- If `m < n`, and `drop m xs = y:ys`, then ...

```
take m (take n xs) = take m xs

y:rest  = drop m (take n xs) = take (n-m) (drop m xs)
        = take (n-m) (y:ys) = y: take (n-m-1) ys
So rest = take (n-m-1) ys

drop n xs   = drop (n-m) (drop m xs)
            = drop (n-m) (y:ys)
            = drop (n-m-1) ys
```

Deriving a more efficient `createTree`

- Substituting appropriately ...

```
go n xs

= (Node tl y tr, drop (n-m-1) ys) where
    m           = n `div` 2
    (tl, y:ys)  = (ct m (take m xs), drop m xs)
    tr          = ct (n-m-1) (take (n-m-1) ys)

= (Node tl y tr, zs) where
    m           = n `div` 2
    (tl, y:ys)  = go m xs
    (tr, zs)    = go (n-m-1) ys
```

Creating a binary tree

- Creating a tree in linear time

```
createTree :: [a] → BTree a
createTree l = fst (go (length l) l) where
    go :: Int → [a] → (BTree a, [a])
    go 0 xs = (Nil, xs)
    go n xs = (Node tl y tr, zs) where
        m          = n `div` 2
        (tl, y:ys) = go m xs
        (tr, zs)   = go (n-m-1) ys
```

Creating a binary tree

- Creating a tree in linear time

```
createTree :: [a] → BTree a
createTree l = fst (go (length l) l) where
    go :: Int → [a] → (BTree a, [a])
    go 0 xs = (Nil, xs)
    go n xs = (Node tl y tr, zs) where
        m          = n `div` 2
        (tl, y:ys) = go m xs
        (tr, zs)   = go (n-m-1) ys
```

- $T(n) = 2T(n/2) + c$ and hence $T(n) = (2n - 1)c$

Showing a binary tree

- To be able to show a binary tree, we need to derive a **Show** instance

```
data BTree a = Nil | Node (BTree a) a (BTree a)
    deriving Show

createTree [0..14] =
Node (Node (Node (Node Nil 0 Nil) 1 (Node Nil 2 Nil))
    3 (Node (Node Nil 4 Nil) 5 (Node Nil 6 Nil)))
7 (Node (Node (Node Nil 8 Nil) 9 (Node Nil 10 Nil))
    11 (Node (Node Nil 12 Nil) 13 (Node Nil 14 Nil)))
```

Showing a binary tree

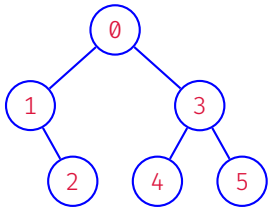
- To be able to show a binary tree, we need to derive a **Show** instance

```
data BTree a = Nil | Node (BTree a) a (BTree a)
    deriving Show

createTree [0..14] =
Node (Node (Node (Node Nil 0 Nil) 1 (Node Nil 2 Nil))
    3 (Node (Node Nil 4 Nil) 5 (Node Nil 6 Nil)))
7 (Node (Node (Node Nil 8 Nil) 9 (Node Nil 10 Nil))
    11 (Node (Node Nil 12 Nil) 13 (Node Nil 14 Nil)))
```

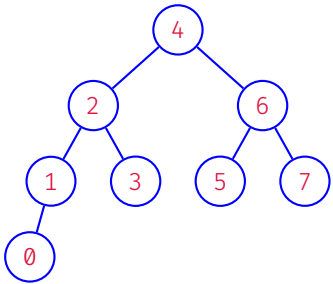
- Not particularly readable!

A custom show



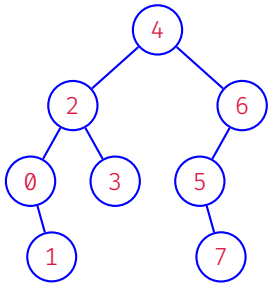

```
[0
  [1
    [*]
    [2]
  ]
  [3
    [4]
    [5]
  ]
]
```

A custom show




```
[4
  [2
    [1
      [0]
      [*]
    ]
    [3]
  ]
  [6
    [5]
    [7]
  ]
]
```

A custom show




```
[4
  [2
    [0
      [*]
      [1]
    ]
    [3]
  ]
  [6
    [5
      [*]
      [7]
    ]
    [*]
  ]
]
```

A custom show

```
instance Show a ⇒ Show (BTree a) where
  show = concat . go 0 where
    go :: Show a ⇒ Int → BTree a → [String]
    go n Nil       = [indent n "[" ++ "*]"]
    go n (Node Nil x Nil) = [indent n "[" ++ show x ++ "]"]
    go n (Node tl x tr)   = [indent n "[" ++ show x ++ "\n"]
                            ++ go (n+2) tl ++ ["\n"]
                            ++ go (n+2) tr ++ ["\n"]
                            ++ [indent n "]"]
    indent n s = replicate n ' ' ++ s
```